

AI-Based Detection of Malicious Open-Source Packages

Evidence-Aware Cross-Ecosystem Triage for npm and PyPI

Revised Project Proposal | Course: Computer Security | Group ID: [Insert Group ID] | Date: [Insert Date]

1. Introduction

Package registries are an important software supply-chain attack surface. A malicious npm or PyPI release can run during installation, access developer credentials, alter files, or contact external infrastructure. Registry scale makes full manual inspection impractical. Detection systems therefore need to prioritize risk while preserving enough evidence for a maintainer to understand and verify an alert.

This project studies an evidence-aware triage workflow for npm and PyPI. It begins with a reproducible static baseline, refers uncertain packages to a controlled dynamic stage, and presents the resulting evidence to a constrained LLM reviewer and a human analyst. The project does not claim that hybrid detection is new. Instead, it measures where a staged policy succeeds or fails: cross-ecosystem static transfer, dynamic-analysis workload, disagreement between modalities, and evidence quality.

2. State of the Art

MalOSS by Duan et al. (NDSS 2021) measured supply-chain attacks across PyPI, npm, and RubyGems. It showed that large-scale package analysis can uncover real malicious releases across multiple interpreted-language ecosystems. Amalfi by Sejfia and Schafer (ICSE 2022) focused on npm and combined machine-learning classifiers with source reproducibility checks and clone detection. Its design demonstrates that post-classification checks can reduce false positives and recover related malicious copies.

Ladisa et al. (ACSAC 2023), the base work for this project, used a shared set of language-independent package features for npm and PyPI. Their released dataset and classifiers provide a reproducible static baseline. The contribution is distinct from MalOSS and Amalfi because it explicitly tests a shared feature space across ecosystems, but static features alone provide no direct observation of installation behavior.

SpiderScan by Huang et al. (ASE 2024) models npm malicious behavior as graphs that include sensitive APIs, control flow, and data dependencies. It uses LLM support to recognize sensitive APIs and invokes dynamic analysis and LLM-based confirmation for selected behaviors. DySec by Mehedi et al. (IEEE Transactions on Information Forensics and Security, 2026) monitors PyPI installation behavior with eBPF kernel and user-level probes. Its 36 dynamic features cover process, file, network, resource, directory, and installation behavior. These works already demonstrate that behavioral and LLM-supported analysis can improve package detection.

LAMPS by Zeshan et al. (Journal of Systems and Software, 2026) uses role-specific LLM agents for package retrieval, file extraction, classification, and verdict aggregation in PyPI analysis. Together, SpiderScan and LAMPS show that LLMs can support code understanding and contextual review. Their results do not make an LLM verdict self-validating: a useful deployment still requires traceable inputs, bounded claims, and a clear route for human review. Gaber et al. (ACM Computing Surveys, 2024) similarly emphasize that dataset realism, feature authenticity, obfuscation, and anti-analysis behavior strongly affect AI malware-detection results.

3. Field-Level Gaps

Comparable cross-ecosystem evidence remains limited. Static models can share a feature space across npm and PyPI, but existing behavioral systems use ecosystem-specific telemetry and representations. This project normalizes source, process, file, and network evidence and reports npm and PyPI results separately, including both NPM-to-PyPI and PyPI-to-NPM static transfer.

The field needs clearer triage-cost reporting. MalOSS and Amalfi support scalable screening, while DySec and SpiderScan use richer but costlier evidence. The project will measure referral rate, analysis duration, missing-trace rate, and reviewer workload rather than assuming that dynamic analysis can run for every package.

Behavioral coverage is incomplete. Install-time traces can miss delayed, environment-gated, dependency-triggered, or previously unseen behavior. Graph matching can also miss behavior outside its known graph set. A clean trace will be recorded as incomplete evidence, not as proof that a package is benign.

LLM use needs auditable boundaries. SpiderScan and LAMPS already use LLMs, so the gap is not the absence of LLM agents. Our LLM component will receive only cited static and dynamic evidence, must identify uncertainty, and will not make unsupported claims about code that was not observed.

Generalization must be tested explicitly. A combined random split may hide poor transfer between ecosystems or new campaigns. The first implemented experiment therefore uses leave-one-ecosystem-out evaluation; future experiments will add time- or campaign-separated labels when provenance is available.

4. Objectives

Reproduce the released Ladisa et al. static baseline on the official npm/PyPI feature dataset and report per-ecosystem performance.

Measure NPM-to-PyPI and PyPI-to-NPM static transfer rather than relying only on a combined test score.

Collect and normalize install-time evidence in an isolated Linux virtual machine for a selected, approved package set, recording collection conditions and analysis time.

Implement an evidence-aware policy that refers uncertain cases to dynamic analysis, flags static/dynamic disagreement for human review, and retains incomplete-evidence limitations.

Use a bounded LLM review prompt to explain cited evidence and compare static-only versus triaged multi-modal outcomes, workload, and false-positive behavior.

5. Dataset and Method

The first dataset is the official `Labelled_Dataset.csv` artifact released with Ladisa et al. It contains 1,940 labelled package records: 1,020 npm records and 920 PyPI records. The file contains the binary label, ecosystem, package name, and 140 numeric static inputs after excluding label and metadata columns. The current baseline uses XGBoost when available and records the split, seed, metrics, and feature importance. The revised code also performs leave-one-ecosystem-out evaluation. In a verified Random Forest fallback run, NPM-to-PyPI achieved $F1 = 0.2016$ and PyPI-to-NPM achieved $F1 = 0.3154$. These directional results motivate, rather than prove, the need for further evidence collection.

The dynamic stage will run only in a disposable Linux Mint virtual machine with no shared folders, clipboard, personal accounts, or host credentials. A trace adapter converts pre-collected strace telemetry into a normalized JSON record containing install hooks, child processes, files read or written, and network connections. The program itself does not execute packages. The decision policy queues static scores in an uncertainty band for dynamic analysis and requires human review when static and dynamic scores diverge. The LLM receives the immutable evidence record and produces an explanation with cited observations, false-positive considerations, and a safe next step.

References

- Duan, R., Alrawi, O., Kasturi, R. P., Elder, R., Saltaformaggio, B., & Lee, W. (2021). Towards measuring supply chain attacks on package managers for interpreted languages. *Proceedings of the Network and Distributed System Security Symposium (NDSS 2021)*.
- Gaber, M. G., Ahmed, M., & Janicke, H. (2024). Malware detection with artificial intelligence: A systematic literature review. *ACM Computing Surveys*, 56(6), Article 148. <https://doi.org/10.1145/3638552>
- Huang, Y., et al. (2024). SpiderScan: Practical detection of malicious NPM packages based on graph-based behavior modeling and matching. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*. <https://doi.org/10.1145/3691620.3695492>
- Ladisa, P., Ponta, S. E., Ronzoni, N., Martinez, M., & Barais, O. (2023). On the feasibility of cross-language detection of malicious packages in npm and PyPI. *Proceedings of the Annual Computer Security Applications Conference (ACSAC 2023)*.
- Mehedi, S. T., Islam, C., Ramachandran, G., & Jurdak, R. (2026). DySec: A machine learning-based dynamic analysis for detecting malicious packages in PyPI ecosystem. *IEEE Transactions on Information Forensics and Security*, 21, 1316-1331. <https://doi.org/10.1109/TIFS.2026.3654388>
- Sejfi, A., & Schafer, M. (2022). Practical automated detection of malicious npm packages. *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*.
- Zeshan, M. U., Ibiyo, M., Di Sipio, C., Nguyen, P. T., & Di Ruscio, D. (2026). Many hands make light work: An LLM-based multi-agent system for detecting malicious PyPI packages. *Journal of Systems and Software*.